

MITOS

keywords: MITOS, MIT, OS

lab-3: pgtbl

1. 在 `proc_pagetable()` 中分配 `usyscall` 的内存，不是一个好主意。影响：失败时，内存页会丢失。
 - Extra: 在 OS 中，思考每个数据结构的所属：数据结构的生命周期，应该被其所属结构管理。
对于 `usyscall` 页面，是对进程的增强，因此其创建和释放应该随着 `proc` 的生命周期处理，而不应该在 `proc_pagetable()` 中被创建（页表的生命周期和 `proc` 无关）。
2. 发现 `panic usertrap` 以及 `sstatus = 0xf, pc=$PC`（缺页）后，我尝试了 `x/i $PC`，发现对应的内存找不到代码——优先考虑是否是栈溢出造成的问题：可能是因为写了没有权限的页面造成的。

lab-5: COW

这个实验很大。8.5 个小时做完的。

值得保持的地方：

1. 了解到 观察-思考 的矛盾：需要增加观察力度；

可以改进的点：

1. 阶段性的总结——先设计好再进行是不太可行的，我是在过程中不断了解怎么修改的；

实验过程

我的实验分成 6 个 commit 完成的。

Step 1.

我大概花了 2 个小时了解情况，做了初步的规划。

要实现 COW，根据文档需要：

1. 实现 `PTE_C`；
2. 能够识别到写中断（`$scause=0xd`）；
3. 子进程能够延迟复制页面；
4. 设置一个保存物理页面引用次数的逻辑；

我看了 `cowtest.c`，有 `simpletest()`，`threetest()`、`filetest()`、`forkforktest()` 四个。我们可以一个个攻克。

我在第一步的时候大概提取到了这样的一些任务。根据任务描述，我们可以入手的地方：

1. `vm.c:uvmcopy()`：任务描述中提到了；
2. `trap.c:usertrap()`：这里可以添加其他类型的中断类型。

这里通过查阅文档，了解到应该参考 `page fault` 相关的原因，在文档中对应的是 `0xd`（写）和 `0xf`（读）。

踩坑 1：我一开始没想明白什么时候需要映射页面。我考虑的实现是：`fork` 的时候应该将页面的 `PTE_R` 设置为 `0`。尝试了这样的设置内核 `panic` 出一个 `0xc`，查表一看是 `page fault` 就用它了。——但很明显这样会让所有的页面（而不只是被写的页面）复制。

按照流程，第一个提交我计划的内容不多：不需要延迟复制，`fork` 时仍然创建新页面拷贝内存；但在页表中 `SET PET_C & RESET PTE_R`；可以检测到缺页，并进入到 `COW` 处理函数、该函数只要复位即可。

第一个 `commit` 实现了上述内容（着重考虑了 `cow()` 函数放置的位置，因为和进程的页面有关，所以放在了 `proc.c`）。由于没有页面需要被多次引用，所以步骤 4 需要的“多次引用逻辑”，暂时没有考虑。

Step 2.

在考虑让子进程延迟复制的时候，我先只在 `cow()` 和 `uvmcopy()` 中改了逻辑，将页面创建的工作放到了前者——此时遇到了问题。分析日志输出发现原因：子进程如果退出，由于在 `cow()` 触发之前父子页面共享一段物理内存，物理页面将在子进程销毁时被销毁——父进程的页面也消失了（具体地，在 `proc_freepagetable()→uvmfree()→uvmunmap()` 中）。

一言以蔽之：共享内存的核心问题 1: Double Free

这个步骤直接设计延迟映射太复杂了，所以我只设计了引用计数的方法，设计了 `pput()` 和 `pget()` 函数——为什么不直接修改 `kalloc()` 和 `kfree()`，让他们自带引用计数的操作？

- 考虑 `kalloc()`：这个操作一定会分配一页内存；我们需要一种方式为某 `PA` 添加计数，和原函数的语义是不匹配的。所以设计了 `pget()`；为了匹配该操作，添加了 `pput()`，但其实 `pput()` 实现到 `kfree()` 中是完全可以的。

我主要针对子进程退出的场景做了修改，相关部分使用了上述设计的两个新的函数；但事实上还有一些隐蔽的地方没有被修改，后续就知道了。

这里的观察：我先做了一点修改，然后观察效果。通过添加日志细节，增大观察到的范围（输出对应日志的地址）；然后找到了我能理解的问题的根源，并做出了修改。其中：

1. 每次一小步是很重要的。做实验就像是在扎根——每一个小步骤都是在更深地扎；但尝试走一步的过程中会遇到很多问题，需要节外生枝地横向探索。如果向深的一步迈得太大，旁支的复杂

度太高，可能就找不到主干、继续深挖的地方了。

2. 观察效果的过程也是在思考，而且也可以迭代：如果当前的信息不足以做出判断，不应该放弃、而是尝试如何获取更多信息。
3. 如果不知道如何获取更多信息，可以开始阶段性的反思，过程中会找到思考的盲点——不过事实上大多数盲点是那些看似不太可能的猜测，在探索的过程中被遗忘了，重新捡回来试试看是好的。

Step 3.

这一步的目标是：实现页面的延迟分配：表现为通过 `simpletest()`。（先不要求父进程也复制页面）

这里我做了两件事：

1. 观察上次修改后 OS 启动速度变慢很多。注意到只有用户的内存页面被映射的时候才需要考虑引用计数，而且 OS 内核在初始化的时候会映射大量界面——都需要过一遍引用计数是浪费的，所以我添加了一个 `vm` 的接口：`uvmmap`，当使用这个接口的时候，会执行 `mappages()`，并自动为映射的页面添加引用计数。（做法：检查使用了 `pget()` 的函数，发现只有 `mappages()`，再检查所有使用 `mappages()` 的地方）
2. 功能实现：一个直观的想法是在 `proc.c:cow()` 中创建页面并复制内容，同时将 `vm.c:uvmcopy()` 中的分配页面和移动的逻辑移除。

这里可能会遗留的点：

1. 在 `cow()` 中，忘记映射之后 `pput()` 原来的页面；
2. 忘记了部分应该使用 `uvmmap()` 的场景：需要的场景有 `uvmfirst()`、`uvmalloc()` 和 `uvmcopy()`：你注意到了，核心在于引用计数，因此有些和 COW 无关的函数也需要修改。

这次修改之后，可以通过 `simpletest` 和 `threetest`。

Step 4.

通过 `filetest()`

这个修改涉及到了多个问题。

核心目标：打开父进程写时复制

这次尝试父进程也打开写时复制吧。主要修改的是 `uvmcopy()`：复制的时候给原来的 PTE 也 SET `PTE_C & RESET PTE_W`。

Copyout 修改

此外还有一个问题之前没有注意到：内核也有可能会访问到用户的未映射的页面。

为什么需要单独处理？

- 答案：内核也会读写用户内存。

之前的修改只考虑了用户态读写用户内存的情况，关键路径是 `scause=0xf` 中断，进入 `usertrap()` 处理；但内核也会读写用户内存，比如这里的 `copyout`。

总结：

- 一般地：考虑修改的触发场景。
- 特别地：COW 的 Write 不止是用户态的写，避免泄露边界情况。

COW 时添加页面引用

修复了个不小的问题：在 COW 的时候，没有对新的页面添加引用。在调试中表现为第三次执行程序时会出现异常。测试用例：

```
echo # Fine
echo # Fine
echo # Error
# usertrap(): unexpected scause 0xd pid=5
#             sepc=0x10ee stval=0x101010101010101
# 后续所有指令都会出现这个错误
```

观察：

1. `scause 0xd`：这是一次读错误
2. `sepc 0x10ee`：不是 `0x8000_0000` 以上的，说明是用户进程的访存错误；
3. `stval 0x....`：这是想要访问的内存地址；

动作：

1. 我在 `pput` 添加了输出 `refcnt < 0` 的时候具体的数字：

```
echo # 2x refcnt = -1
echo # 3x refcnt = -2, 1x refcnt = -2
echo # Crash, but with 3x refcnt = -1
```

我无法完全还原现场.....不过可以看出来是忘记 `pget()` 了，应该能解决问题吧.....

Step 5.

通过 `forkforktest()`

修改不大：将 `refcnt` 数组放到了 `kmem` 结构体，共用了锁避免竞争问题。

Step More

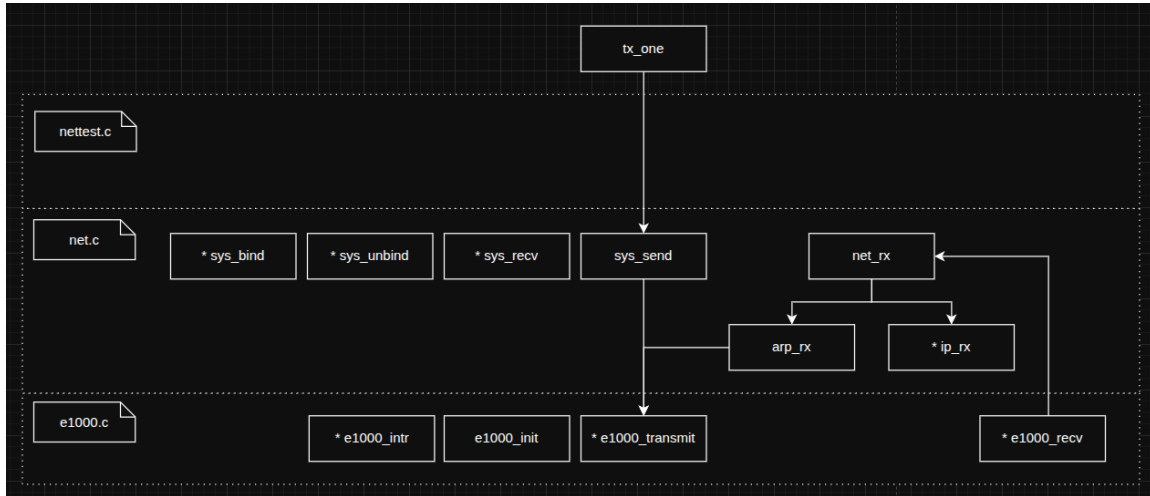
后面的步骤做了一些重构、和微小的 BUG 修复。

lab-6 NET

任务开始前的准备工作

看完 xv6 手册后，我花了 1:20 做了：

1. 过了下代码的结构：



2. 看了实验的要求。

我注意到实验分成两个部分：

✓ Part 1: 实现 `e1000_transmit()` 和 `e1000_rcv()`。这个任务又分成：



1. 实现传输：实现 `e1000_transmit()`；



2. 实现接收：实现 `sys_rcv()` 和 `e1000_rcv()`。

✓ Part 2: <暂时没看>

我可以先从 实现传输 开始。可用资源：

1. 如何测试效果？

1. 目标：make grade 中通过第一个；
2. 查看传输包：tcpdump -XXnr packets.pcap
3. 先启动 python3 nettest.py txone，然后 nettest txone；

2. 设备的细节：探索手册！需要看：

1. 设备概览：Section 2；
2. 传输：Section 3.3；
3. 寄存器：Section 13 (Optional)；
4. 设备如何初始化：Section 14 (Optional)；

3. 代码：

1. `sys_send()`；

2. `net.c/arp_rx()` 中可以查看使用 `e1000_transmit()` 的时机 (Optional, 因为这条链路主要是接收包时用的)

4. 网页:

1. 先看看 Part 1 的描述, 有设备的设计思路; 然后查看手册;
2. 遇到不懂的, 就看下 [hint](#);

这个任务是 信息收集型 的。比起思考, 更需要了解如何收集、组织、使用信息。
やるな!

Part 1.1 传输

任务:

- ✓ 阅读 `sys_send()` ;
- ✓ 看看 Part 1 的描述, 有设备的设计思路;
 - ✓ 阅读 `e1000_init()` (Optional)
- ✓ 查看手册: Section 2、3.3;
- ✓ 实现 `e1000_transmit()` ;

Hint:

- 查看传输包: `tcpdump -XXnr packets.pcap`
- 先启动 `python3 nettest.py txone`, 然后 `nettest txone` ;
- 遇到不懂的, 就看下网页的 [hint](#);

过程:

1. 我先阅读了 `sys_send()`: 用 `eth`, `ip`, `udp` 三层结构体包装了数据并给到 `e1000_transmit`, 内容简单;
2. 阅读了 Part 1 的描述: 我应该阅读 `e1000_init()`, 了解如何操作 TX-Ring. 0. 额外讯息: 记得释放 `buff`, 无论成功与否。
 1. `e1000.c` 中的 `tx_ring` 和 `tx_bufs` 是我要关注的内容;
 2. `regs` 和 `e1000_lock` 可能会被经常用到。前者需要额外了解; 76 行开始的 `transmitter control bits` 需要我额外关注。
 3. `e1000_init` 中也有关传输的数据结构, 后续可以查看。
3. 我认为 `e1000_transmit` 需要:
 1. 写入 TX;
 2. 调整寄存器 (我需要额外查看 `e1000_defs.h`);
 3. 配置好 `e1000_intr`, 响应发送后的中断 (Optional);
4. 我开始尝试编写代码。
 1. 我注意到需要了解 `tx_desc` 的结构。这里的注释指引我查看文档 3.3.3; 阅读后我知道如何设置 `tx` 了;

2. 然后我回来看了 Part 1 的描述，了解到想要通知硬件发送包，需要配置寄存器；了解到寄存器相关代码的编写方式，并了解到需要下一步查看手册中寄存器相关的部分；搜索 Part 1 提到的 `E1000_TDT` 寄存器，找到了手册 3.4；
3. 我阅读了手册的 3.4 的内容，里面给出了 `TX_RING` 的结构、相关寄存器的解释。我尝试在设置 `tx_desc` 后将 `regs[E1000_TDT]` 自增 1，然后试着运行了下 `nettest txone`，接收端成功收到了包。此任务完成。
4. `buf` 没有释放的问题后续解决。

注意查看这个步骤的 `commit`，其中的 `kernel/e1000.c:e1000_transmit()` 中包含了一些有价值的注释。

我学到了什么：

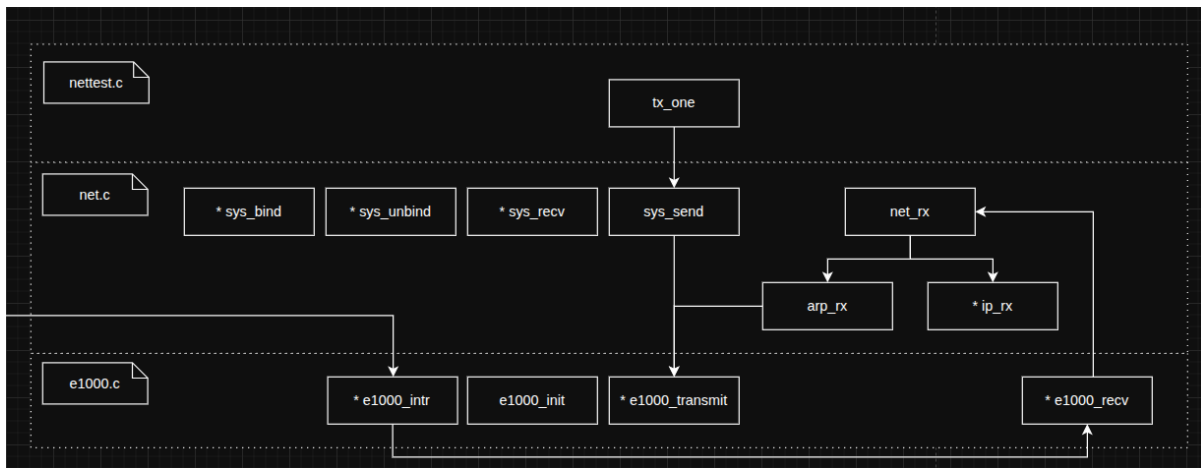
- 形而下的：
 1. 了解到 `tx_ring`、以及如何使用寄存器；
 2. 过了一遍发送包的流程：了解到如何从 `send()` 到 `e1000_transmit()`；`sys_send` 中需要准备好各个栈的包头，并将内存交给下一层，`e1000_transmit` 则需要和硬件打交道，了解到需要用到的数据结构；
 3. 了解到 `TX_RING` 是怎样运作的。
- 形而上的：
 1. 任务的分解：拿到这个问题后我先探索了部分问题结构：形成了一个当下的任务清单；可以看到不是所有的任务都搞清楚了再做；我还绘制了一张结构图，图中标记了指南提到的所有函数，这让我能直观地知道我应该从哪里开始；
 2. 列出可用的资源：我先将所有的材料都列出来了，这样当工作做到一半不知道如何推进时，就可以考虑从哪里寻找答案；以及，我尝试给每个资源都列出了可能使用的时机，很有帮助；
 3. 指路和询问：别人的一个提示，可以作为关键词搜索。在我不知道设置了 `rx_desc` 设置好后要怎么做时，指南中的 `E1000_TDT` 给了我在了手册中搜索的线索。
 4. 如何查阅手册：不要全面地看。我带着“实现传输能力”的目的来看，就知道了应该查阅手册的哪些部分了。

Part 1.1.1 解决内存没有释放的问题

重新读了下手册（以 `free` 为关键字），注意到可以在写入 `tx_ring` 之前判断地址是否不为空——不为空说明之前用过，此时释放就可以了。`Tx_ring` 中会驻留最多 `TX_RING_SIZE` 个页面，不会损失太多。

Part 1.2 接收

我简单读了下相关的代码（从 `e1000_recv` 开始），我为结构图添加了一些内容：



框外的箭头来自 usertrap.c

我做了：

1. 走读代码后注意到 `e1000_recv()` 是我要实现的。先前的知识提到它应该调用 `net_rx`；
2. 阅读了 指南，正常接收到 `python3 nettest.py rxone` 的包，内核会自动输出两行日志；
3. 查看了 `nettest.c`，看来还需要和 `sys_recv` 对接。

任务：

- ✓ 阅读 `arp_rx` 和 `net_rx`，确定这两个函数处理的工作的范围；
- ✓ 在 `e1000_recv()` 中处理 `RX_RING` (猜测)；
- ✓ 阅读手册、指南相关内容：Section 3.2 是重点。
- ✓ 实现 `ip_rx`；
- ✓ 实现 `sys_recv`：可以参考 `sys_send`。

上一个任务顶多代表我看懂手册了—这个任务真正接手设备驱动的工作了。

我是这样做的：

1. 我读了下 `arp_rx` 和 `net_rx`，了解了 `ip_rx` 实现的一部分逻辑；
2. 我在尝试实现 `net_rx` 的时候意识到需要有一种方法和 `sys_recv` 通信—我去阅读了 `console.c` 的内容，知道了应该使用 `sleep` 和 `wakeup`。

在上述两个步骤后，我意识到 `ip_rx` 需要两个部分：

1. 解包并获取 `buffer` 中的内容；
2. 设计数据结构，用于将调用 `sys_recv` 的进程挂起，以及在 `ip_rx` 中唤醒—这需要一个地址 (参考 `console.c`)。

不过我需要先实现 `e1000_recv()` 再说。

3. 我先阅读了 手册关于 `RX_RING` 的部分 (3.2.6)，然后实现了 `ex1000_recv()`。可以正常调度到 `ip_rx`；

4. 我阅读了 `console.c` 的设计，在 `ip_rx` 和 `sys_recv` 中添加了 `sleep` 和 `wakeup` 的机制——前者收到数据包时，后者可以被唤醒了。

- 但两者之间还不能交换数据。`sys_recv` 需要知道：
 1. 发送方的 IP 和地址；
 2. 发送方的数据；
- 判断端口：只有发送到发送方指定的端口的数据包，才应该被对应的 `sys_recv` 进程处理。

我认为我应该：

1. 看看 `hint` 是否有提示；
2. 设计一个数据结构（如 `console.c`），以接收方的端口为键，设置 `sleep` 的地址；
3. 对应的地址的数据结构应该能起到传输 `sys_recv` 需要的数据的目的。

这个任务有些复杂，需要单独开一个子任务。不过看指南的时候发现其实现在已经完成了 Part 1 的任务了——上面提到的都是后续 Part 的工作，而且现在的代码已经可以通过指南中提到的 `make grade` 前两个测试用例了。我们进入下一个部分。

现在的状态：

1. 手动测试 `nettest txone` 和 `rx` 都 OK；
2. `make grade` 通过前三个测试点。

Part 2

首先我看了下指南，发现提到的内容我之前基本上都有了解到了。所以这里主要是如何安排任务的顺序的问题。

另一个查看的是所有可用的测试用例：

```
Usage: nettest txone
       nettest tx
       nettest rx
       nettest rx2
       nettest rxburst
       nettest ping1
       nettest ping2
       nettest ping3
       nettest dns
       nettest grade
```

1. 我想先从上一个任务想要完成的任务开始：实现 `nettest rx` 顺利通过。这会让我打通从 `e1000_recv` 到 `ip_recv` 再到 `sys_recv` 的链路，也实现了 UDP——而这正是这个任务想要我做的。
2. 然后，我应该考虑设计存放 UDP 包的数据结构；

3. 其次是 `bind()` 相关的逻辑；这个完成后，我应该可以通过 `rx` , `rx2` 和 `rxburst`

后续可能有这几个任务需要我做：

1. 多个包一起来时，我应该可以正确地丢弃包；
2. 解决回绕问题—通过 `ping` 的三个测试用例。

Step 2.1 完善数据传输 (`nettest rx`)

这个步骤中，我设计了 `ports` 结构体，其中包含了被缓存的页面，并让 `sleep` 以它为目标。这里的设计不是很难，但排查了很多错误，包括：

1. `sys_recv` 的长度问题：没有注意到要返回被复制的长度；注意到后：
 1. 尝试将来自 `e1000` 的 `len` 直接保存（原本的设计中 `ports` 包含了这个长度以让 `sys_recv` 获取到）：`e1000` 接收到的包会在末尾有填充，根据该长度减去各包头的大小，并不能得到正确的结果；
 2. 直接使用 `udp->ulen`：没有注意到这个长度包含了包头；减去包头后解决了问题。
2. 字节序的问题。

都是小问题。

通过了 `nettest rx`（发现需要运行 `python3 nettest.py rx` 而不是 `rxone` 通过此用例）

Step 2.2 实现 `bind()` 和处理多个包的问题

修改任务内容。

- ✓ 实现 `bind()`
- ✓ 实现每个端口的 16 个包的缓存；—
- ✓ 在 `e1000` 层，处理回绕问题。（这个问题不存在）—
- ✓ 在 `e1000` 层，处理一次中断多个包的情况；—

此阶段完成了 1 和 4。其他的是后续完成的。

1. 我向 `ports` 数据结构添加了 `pid` 字段，以记录每个端口都什么进程占用了；然后实现了 `sys_bind` 和 `sys_unbind`，并修改了 `ip_rx` 以丢弃没有使用的内存。此外，我在 `e1000.c` 中添加了一次处理多个包的情况。现在通过了 `rx2`；
2. 我检查测试用例，又正常通过了 `ping0`，所以我的下一个任务是通过 `ping1`；这里的 `ping` 总是会在中途卡住。我认为可能的原因和尝试有：
 - 也许包没有发送出去？→ 我在 `e1000_transmit` 中添加了输出，发现没问题；
 - 也许是接收侧的问题？→ 我猜测可能是 `ip_rx` 没有通知到 `sys_recv`，但通过添加日志发现可能是这个问题—`ip_rx` 收到了包，但 `sys_recv` 没有推进。
 - 我推测可能是 `netlock` 的问题：可能哪里忘记释放了。我尝试添加一些输出，找到了另一个可能的原因—也许是 `sleep` 没有被唤醒？

- `sys_recv` 的 `sleep` 晚于 `ip_net` 的 `wakeup`——上一点添加的输出日志帮我总结出了这个原因。

所以后续我在解决这个问题。我注意到两个地方没有考虑到：

- `ip_rx` 中，赋值 `buf` 时没有加锁（主要原因）；
- `sys_recv` 中，`sleep` 之前没有判断是否已经有包被缓存了（主要原因 2）；

解决了这两个问题后，正常通过了 `ping1`。

经验：

1. 需要竞争的数据结构，修改前一定要记得加锁。
2. `wakeup` 在 `sleep` 之前导致进程没有被唤醒是一种常见的错误。这是一种典型的生产者-消费者问题，核心关注数据是否准备好。通过：
 - `sleep` 前先判断是否需要 `sleep`（数据真的没准备好吗）；
 - `wakeup` 和 `sleep` 外加锁（保证竞争的数据结构）；是解决此类问题的常见思路。

Step 2.3 通过 `ping2`

简单分析了下没通过的原因。`ping1` 和 `ping2` 都是循环发送若干个包并接收，但不同在于：

1. `ping1` 是循环内部，每次发送后接收；
2. `ping2` 是，先循环发送所有包，然后循环接收所有包。需要有缓存机制可以避免丢包。

这个问题想要实现，有很多要点。其中最重要的是设计一个队列，而且要处理好溢出的情况。

我实现后发现问题主要在考虑清楚队列要如何设计，以及 `wakeup` 和 `sleep` 的时机。事实上，我一次就把这个任务搞定了（我说不定很强哦，哈哈）。现在已经能通过除了 `nettest free` 之外的所有任务了。

接下来的任务有：

1. 找到可能有内存泄漏的地方；
2. 重整代码，更好地抽象添加的 `ports` 数据结构；
3. 搞定进程异常退出时，可以正常 `unbind port`；

Step 2.4 找到内存泄漏的原因，通过最后一个测试点

看了下测试用例，只要不要丢失超过 32 个页面就可以了。

我基本上没有做修改。我注意到我手动运行测试用例的时候，有时候会卡在 `dns` 上。原来这个 `dns` 这个测试用例需要你的宿主机连接到互联网才能正常通过。我连接到网络，并且将测试用例的超时时间改为 60（在 `grade-lab-net:17`），最后这个样例也顺利通过了。好耶！

Part Extra. 重构 + 总结反思

简单重构了下。这个任务我用了 13 个小时，远超要求的 4 个小时。

遗留问题 + 反思

- 形而下的：
 1. 超时：程序没有在 30 秒内完成。估计原因是和加大锁有关；
 2. 大锁：可以每个 port 搞一个小锁，避免太过分的竞争问题；
 3. 上半部中加锁：中断的上半部 (i.e. ip_rx) 的部分竟然用到了自旋锁！通常不会这样做——但我查看 `console.c:consoleintr()`，直接给整个流程加锁了，所以也许没问题——但之后应该注意尽量避免使用锁。
- 形而上的：
 1. 耗时太长：4 个小时的工作用了 13 个小时。可能的原因有：
 1. (pros) 记录花费了比较多的时间：这次记录了全过程；
 2. (pros) 我计划工作内容花费了一些时间(包括绘制层次图)，但让整个实验流程可控；
 3. (cons) 我整理已有信息的能力有待提高：开始做之前就已经花了一个半小时了。
 2. 这个任务没让我搞清楚什么时候、应该问别人什么问题，来获取更多信息。

收获

- 形而下的：
 1. 对锁的使用浅尝辄止：了解到了 wake before sleep 问题，以及什么时候加锁、数据竞争的初探；
 2. tx_ring, rx_ring 是怎么回事儿；
 3. 了解了上半部和下半部；
- 形而上的：
 1. 我敢于查文档查材料，并知道了带着需求看目录查阅材料的重要性；
 2. 我相信自己可以做好信息搜集型工作，并能做得比较出色；
 3. 我实践了一次 工作流。知道了 信息收集-目标分解-流程-可用资源-验证方法 是一个不错的范式
 4. 形成图像：drawio 挺好用的，整理了信息画成结构图，后续可以参考。
 5. 我注意到，搜集信息比内耗思考能更好地解决问题；
 6. 如果发现思考已经无法推进问题了，那就应该换个方向了：以获取信息为目标，思考应该获取怎样的信息。← 我需要增强这方面的能力。

其他收获可以看：[^4d1112](#) 和 [^2bb7e3](#)

lab-7 Lock

书的这一章节写得不错。读完后我了解到：

1. 锁，用来保证 “invariant”：我们对数据结构的状态有一些假设，锁保证不会在这些假设被打破的中间情况下，被其他 CPU 影响到执行（链表添加新节点需要赋值两个指针，中间赋值一个的时候就是一种 invariant）；
2. 非 invariant（非稳定的状态）被打断，是可能引发问题的原因。锁本来用于避免 invariant 情况（程序可能在数据结构不稳定时被打断），但要特别注意其他可能会打断的情况！比如：
 1. 可重入锁：f 和 g 都有持锁的临界区，但如果 f 在临界区中调用了 g（且使用了可重入锁），则 g 可能会看到 f 执行了一半下的不稳定的数据结构；
 2. 中断：如果 f 执行了一半时被中断并执行了中断处理函数 g，则 g 也会看到 f 执行一半下的不稳定的数据结构。（解决方案：获取锁时关中断）
3. 内存屏障应该写在锁操作的边缘，避免临界区内外的代码“渗透”到对面（避免指令重排）。

此外，我还获取了这些信息：

1. 根据指南，一共两个任务，而且不相关。我可以先从第一个任务开始，这是一个将一个内存分配改成多个的工作；
2. 除了读书的锁这一章，还有其他章节需要我读。一会儿扫一下；
3. 锁里包含了 n 和 nts 两个额外字段，分别代表了尝试获取锁的次数，以及实现中自旋的次数。
4. 指南说，尽量避免让机器运行其他任务。
5. 我需要看看 kallocetest，来确定到底在测试什么。
6. 还没有看 hint，一会儿可以参阅。

Step 1 重新设置内存分配

任务 1：重新设计内存分配

- ✓ 看看 kallocetest，了解日志输出的含义；
- ✓ 了解 bache 的部分是否是这个任务需要的；
- ✓ 设计一个并发的分配方法；
- ✓ 参阅 hint。

你的任务是实现每个 CPU 的空闲列表，并在某个 CPU 的空闲列表为空时进行窃取。你必须为所有锁命名，名称以“kmem”开头。也就是说，对于每个锁，你应该调用 `initlock`，并传入一个以“kmem”开头的名称。运行 `kallocetest` 以查看你的实现是否降低了锁争用。为了检查它是否仍然能够分配全部内存，请运行 `usertests sbrkmuch`。你的输出将类似于下面所示，kmem 锁上的总争用大幅减少，尽管具体数值会有所不同。确保 `usertests -q` 中的所有测试都通过。 `make grade` 应显示 `kallocets` 通过。

检查了下代码，在 `kernel/spinlock.c:snprint_lock()` 中找到了输出信息的含义：

```
snprintf(buf, sz, "lock: %s: #test-and-set %d #acquire() %d\n",
          lk->name, lk->nts, lk->n);
// lock: kmem: #test-and-set 0 #acquire() 94703
```

这行日志代表了：kmem 锁，尝试获取了 94703 次锁，但没有一次是被自旋的——我们应该尽量减少第一个数字的大小。

这个任务比较简单。我使用了比较简单的策略：一开始将所有的页面分配给 cpu-0，之后如果遇到了某个 CPU 的 freelist 为空，则从其他任何一个有页面的链表中偷取一页——这个策略可以顺利通过 allocatest 的 test 1 和 2，但 3 会遇到冲突太多的情况（total = 19428，要求在 10000 以下）。

```
== Test    kallocatest: test3 ==
kallocatest: test3: FAIL
...
lock: proc: #test-and-set 16704 #acquire() 3325661
lock: kmem-1: #test-and-set 14875 #acquire() 1446429
tot= 19428
test3 FAIL m 2 n 19428
$ qemu-system-riscv64: terminating on signal 15 from pid 600746 (make)
MISSING '^test3 OK$'
```

因为只是个实验，所以用点 tricky 的方法好了：每次窃取 8 页（或者不足 8 页时，全部窃取）。——失败了，而且比原来更离谱了（原本是 20k 左右的冲突，现在干到 30k 了）。这说明一次分配 8 页反而会激化冲突。

我想应该设计一个计数的方法：选择剩余页面最多的，从它折取一半。

又写了一版。第二天来写，精神状态好多了，思路也清晰了。这次我重写了“偷一页”和“偷一半”的逻辑，在测试偷一页。测试完成后：

1. 如果没问题：
 1. 再测试下另一种是否 OK；
 2. git rebase: 比较下和之前的区别，为什么这次 OK 了；
 3. 看下一个任务中和 bache 相关的部分；
2. 如果有问题：
 1. 看看这次的总计冲突次数，是否比之前会有降低；
 1. 如果降低，说明这次重构是可行的.....和之前的比较下；
 2. 同时可以确定另一个方案大概率能过。试试看。
 3. 如果没有降低.....？
 2. 试一下另一个方案。
3. 如果另一个方案通过了一：
 1. 那就 git rebase 比较下；
 2. 然后看下一个任务；
4. 如果没有通过，再规划。

下一个任务需要我：

1. 先看下书的 8.1~8.3 节；
2. 阅读 kernel/bio.c；

3. 重新阅读指南和提示；

完成。做了一个小修改后，只偷一页的方案就通过了。这个实验用了 4 个小时—2 倍，平平无奇。

窃取一半的方案没有成功—猜测原因是数量为偶数时，窃取时是向下取整的。当内存紧张时，对于三个进程的情况，进程 A 可能看到 B 和 C 各有一个页面，但无法获取，导致空转。

暂时先不做，进入下一个实验了。

Step 2. bcache

这个任务 15:00 开始进行。

读完教程，有个初步的设计了：设计 14 个桶：前 13 个用作哈希，最后一个作为空桶。不用双向链表，而是单向的。

我打算先修改代码，能让一个 CPU 的情况正常执行；然后再考虑下一步工作。

Step 2.1 bucket

我编写了使用 hash 和 bucket 的代码—但速度太慢了，所以没有通过测试用例；也没有完美降低获取锁失败的次数。我想问题在于，每次 bget 和 brelse 我都要尝试操作 freelist。我想，可以给每个进程一个缓冲的空间：release 的时候，可以给自己的 bucket 缓存一个空 buf(仅一个)；在 buf 紧张时，可以被夺走。

这应该是下一步的工作了。

Step 2.2

我做了一些修改.....具体设计看代码，里面的注释很清晰。

但问题是：为什么运行了 test0 就会让 test3 失败？我在排查问题。

Shit！原来是我错误地赋值，导致遗漏了 12 / 30 个 buf！改掉之后就可以通过所有测试用例了。

不过我注意到还是超时了。重新读了代码后发现，我的设计让缓存完全没有用到（我错误地理解了原来的代码）。我调整了下思路，将 free 字段也纳入到考虑中。这次看上去系统变得没有那么卡顿了。满分通过。

这个部分我用了 5 个小时完成的。总计花费了 10 个小时。

lab-8 FS

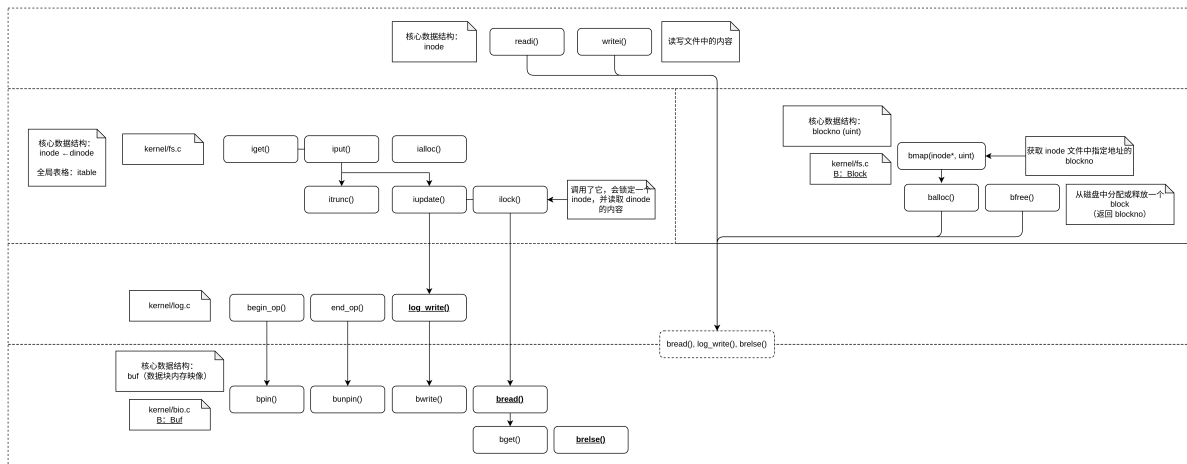
这个任务需要我花很多时间阅读—我之前对文件系统接触得不是很多。

我花了 4 个小时阅读书的这部分内容，难顶，我学到了一些内容，但想要全面地理解仍然很困难—主要原因是概念的数量比较大，而且涉及很多锁。

主要涉及的文件： bio.c , fs.c , sysfile.c , log.c 。重新读一下试试。

- ✓ bio.c
- ✓ fs.c
- ✓ sysfile.c
- ✓ log.c

又花了 1 个小时将几个文件都看过了后，整理出下面的结构图：



我们先从第一个任务开始。

Task 1. 添加 doubly-indirect block

核心是修改 bmap，但同时也要修改 inode 和 dinode。排查后，需要做修改的有这些：

- ☒ bmap()
- ☒ +writei()
- ☐ +readi()
- ☐ inode
- ☐ loadseg
- ☐ exec
- ☐ dinode
- ☐ ialloc
- ☐ iupdate
- ☐ +ilock, iunlock
- ☐ +itrunc
- ☐ mkfs:main
- ☐ +iget, iput
- ☐ idup
- ☐ +stati
- ☐ +dirlookup

- ☐ +dirlink
- ☐ namex, namei, nameiparent
- ☐ bunch of functions in sysfile

我应该从 `bmap` , `writel` , `readi` , `iget` , `iput` 四个函数开始。

1. 先让 `bigfile` 通过;
2. 然后通过所有的 `usertests`。

这个实验真心不难，基本上是一次过了。这次在时限范围内——1.5 个小时。

虽然可以再写得好看一些（比如抽取抽象，从“一页”inode 里查找某个 `blockno` 的），但对于这种不太用动脑子的任务，还是不浪费时间了。看看下一个了。

可能和机器性能有关，我的 `bigfile` 无法在 240 秒内完成，但可以在 300 秒内完成任务。这里就稍微调整下测试用例了。

Task 2. Symlink

先把系统调用需要添加的 样板代码 添加好了；然后需要考虑修改哪些函数。

可以从系统调用入手(`kernel/sysfile.c`)：

- ☐ `sys_fstat`
- ☐ `sys_unlink`
- ☐ `create` (重点参考)
- ☐ `sys_open` (重点参考)
- ☐ `sys_mknod`

可以从 `sys_open` 开始，看看是怎么通过一个文件名（或者 `fd` 号），得到一个 `inode` 的。

这个任务的目标是可以：

1. 创建一个足够好的 `inode` 映像；
2. 保证 `sys_open` 的时候，可以根据这个映像，找到合适的 `inode`；
3. 可以正常地 `sys_slink` 到新文件；可以正常地读到新文件；
4. `unlink` 时可以删除掉新文件；
5. 如果原始文件被删除了，无法打开链接的文件；
6. 如果原始文件不存在，也可以创建链接——但等原始文件存在后才可以读取；
7. 可以多级链接；
8. 打开的 `link` 的数量可以超过 `NINODE + 2` ？
9. Extra: 保证权限是 OK 的。

Step 2.1 可以顺利创建链接文件

- ✓ 可以创建文件；—
- ✓ 可以正确读取链接文件的内容；—

Step 2.2 处理循环 link 问题

- ✓ 处理循环 link 问题
- ✓ 解决多个链接同时存在的问题

3 个小时完成。

总体而言，这个实验的难点在于理解文件系统各个部分的功能，而不在于建立连接和深度接触。我从这个实验中学到的并不多——或者说，可能是我已经入门了吧。

lab-9 mmap

本实验 15:00 开始。

我自己尝试分析了下这个任务需要的是什么；我看了指南和 Lecture，现在我对如何实现有 60% 的理解了。

我注意到做这个实验的时候我思路很清晰。除了对这个实验的内容比较了解，还有一个原因是，我能找到所有我需要的信息。我也知道自己应该问什么问题。

我认为我应该增强知道应该问什么问题的能力。

此外，将任务分解成足够小的步骤，也是我应该做的；我需要时刻知道自己处于怎样的位置上。

第一个任务：能实现映射进来、能读到、能写到就算成功。

Step 1 Simple Read & Write

Step 1.1 Simple Read

- ✓ 只要让文件可以被映射进来，并能正常读取即可。—
- ✓ 正常地执行简单的 `sys_munmap`；—

Step 1.2 Simple Write (Private)

- ✓ 支持 Private 的写入
- ✓ 支持分段的分页载入，而不是第一次读取时就将整个文件读入。—
- ✓ 支持映射多个文件，使用新设计的文件映射布局；—
- ✓ 支持对部分 vma 的解映射；—
- ✓ 支持对非整页部分内存的看护；—
- ✓ 解决：找到了相同的文件，但是用不同的权限映射；—

一共 15 个小时，完成了这个实验。

总结：所有收获

- [COW 实验中，关于观察和探索的思考](#)：形而上的
- [NET 中，关于问题分解、知识地图的思考](#)：形而上的
- [Net 中，整个实验的收获](#)：形而上的
- [Lock 实验中，读书得到的收获](#)：形而下的

分支名称	用时 （小时）
util	6
syscall	8
pgtbl	9
traps	5
cow	9
net	13
lock	10
fs	5
mmap	15

总计花费 80 小时。

| 注：此文件来自 obsidian vault: note